

“Inventory and Warehouse Management System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the
degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

M. Laasya Sri **2520090196**

M. Rajini **2520090197**

K. Ramya **2520090313**

Section 12 Team 17 CS-IT

Under the Esteemed Guidance of

Dr. Kaja Shareef

Head of Department

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “Inventory and Warehouse Management System” is a record of bonafide work of M. Laasya Sri - 2520090196, M. Rajini - 2520090197 and K. Ramya - 2520090313 submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 12, Team 17, during 2026-2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

M. Laasya Sri - 2520090196

M. Rajini - 2520090197

K. Ramya - 2520090313

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled “Inventory and Warehouse Management System” is a bonafide work done and submitted by M. Laasya Sri - 2520090196, M. Rajini - 2520090197 and K. Ramya - 2520090313 in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 12, Team 17, at KLH University during 2026-2027, Trimester 4.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who assisted us in one way or the other in completing this project.

We express our gratitude to Dr. Kaja Shareef, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

Our greatest appreciation goes to our Course Coordinator and project guide, [Guide Name], Department of Computer Science and Information Technology, for their support, encouragement and guidance throughout this project.

We thank all the members of the teaching and non-teaching staff who assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

M. Laasya Sri - 2520090196

M. Rajini - 2520090197

K. Ramya - 2520090313

Abstract

The Inventory and Warehouse Management System is a web-based application designed to efficiently manage products, warehouses, suppliers, staff, stock transactions and orders through a centralized platform. Manual stock registers, scattered spreadsheets and unclear storage locations lead to stock mismatches, wasted warehouse space and slow product retrieval. The system provides a secure and organized environment for monitoring inventory, tracking stock movement, managing warehouse capacity and viewing the products stored in each warehouse.

The proposed system offers a responsive interface for secure login, product and supplier management, warehouse and storage-location management, stock-in, stock-out and transfer transactions, order handling, barcode-based product lookup and dashboard charts. Authentication and role-based access control protect the system, so that each user can reach only the functions that belong to their role.

The system also focuses on improving warehouse efficiency through Warehouse Slotting Optimization. DSA techniques such as sorting, priority queues and greedy algorithms are used to prioritize products based on demand and assign them to suitable warehouse locations, considering storage capacity, product requirements and accessibility. This improves inventory visibility, utilizes warehouse space efficiently and reduces the time and distance required for product retrieval.

The application uses React.js with Chart.js for the frontend, Node.js and Express.js for the backend, and MySQL for storing related records. The documented database has eleven tables covering users, staff, warehouses, storage locations, categories, suppliers, products, inventory, stock transactions, orders and order items. Transactions and row locks support consistent stock updates, and JWT secures protected requests. This report describes the planned design and features of the project; execution outcomes must be confirmed during the project demonstration.

Index

S.No.	Chapters	Topics	Page No.
	Acknowledgement		4
	Abstract		5
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	7
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	9
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	10
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	11
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Design Decisions and Assumptions	12
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	14
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	16
8	Testing	8.1 Postman API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	17
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	18
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	19
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	20
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	21
13	References	Documentation, tools, and resources used	22
14	Appendices	Sample Code Snippets Installation / Setup Instructions Default Login Credentials API Reference Summary Screenshots (to be added by the student)	23

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Warehouses hold the stock on which retail, manufacturing and distribution businesses depend. In many small and medium warehouses, stock is still tracked in paper registers or separate spreadsheets, supplier details are kept elsewhere, and the physical location of a product is known only to the staff who placed it. As the number of products, suppliers and orders grows, stock counts drift from reality, products are stored in awkward locations and staff spend extra time walking to find items.

The Inventory and Warehouse Management System brings products, warehouses, suppliers, staff, stock transactions and orders into one database-backed web application. Authorized users can see what is stored in each warehouse, record every stock movement and view summaries on a dashboard. A Warehouse Slotting Optimization module suggests where products should be stored so that frequently requested items are easy to reach.

1.2 Problem Statement

The project addresses the difficulty of managing products, warehouse space, suppliers, stock movement and customer orders through disconnected manual processes. The system must keep stock quantities accurate, stop stock from being issued when it is not available, respect the capacity of each storage location, protect data with role-based access, and help the warehouse place fast-moving products where they can be retrieved with the least time and distance.

1.3 Scope of the Project

The scope includes secure login with role-based access, management of products, categories, suppliers, warehouses, storage locations and staff, recording of stock-in, stock-out, transfer and adjustment transactions, order creation and tracking, barcode-based product lookup, dashboard charts and Warehouse Slotting Optimization. React, Express and MySQL form the planned web implementation. Hardware integrations such as RFID readers or automated storage systems, and a dedicated native mobile application, are outside the current scope.

1.4 Objectives

- Maintain a central, accurate record of products, suppliers, warehouses and storage locations.
- Track every stock movement (stock-in, stock-out, transfer, adjustment) so that quantities can be traced and audited.
- Monitor warehouse capacity and show the products stored in each warehouse and location.
- Keep stock updates consistent through database transactions, validation and constraints, so that stock never becomes negative or exceeds location capacity.
- Secure the system with JWT authentication and role-based access control.

- Apply sorting, priority queues and greedy algorithms to assign high-demand products to accessible, suitable locations.
- Provide dashboard charts and barcode lookup that help staff and managers act quickly.

1.5 Importance of the Application

By centralizing inventory data, the application reduces manual record keeping and improves visibility of stock levels. Managers can see low-stock products and warehouse utilization at a glance, staff can look up a product by scanning its barcode, and the slotting module reduces unnecessary travel inside the warehouse. A transaction history gives accountability for every change to stock.

1.6 Target Users / Audience

The system serves warehouse administrators, warehouse managers and warehouse staff. Administrators manage users, warehouses and master data. Managers monitor stock, orders and reports and run slotting optimization. Staff record stock movements, scan products and fulfil orders for the warehouse they are assigned to.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

A development computer with a dual-core processor, at least 4 GB of RAM and sufficient free storage for Node.js dependencies, MySQL and the project files can run the application locally. An 8 GB RAM system provides a more comfortable development environment. Users need a modern browser on a computer, tablet or phone. Barcode lookup requires a camera-enabled device or a USB barcode scanner; products can also be found by typing the barcode or SKU manually.

2.2 Software Requirements

- Node.js 18 or later, with npm
- MySQL 8.0 or later
- A modern web browser (Chrome, Edge or Firefox)
- Git, for version control
- MySQL Workbench and Visual Studio Code for development (optional but recommended)
- Postman, for API testing

2.3 Development Tools and Frameworks

React.js supports the frontend and Chart.js draws dashboard charts. Node.js and Express.js provide the REST API. MySQL stores relational data through the mysql2/promise driver. JWT supports authentication, and a barcode scanner API supplies camera-based barcode reading in the browser. VS Code and MySQL Workbench are used for development; Postman supports API validation.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend uses React.js with reusable components for the dashboard, product tables, warehouse views, stock-transaction forms and order screens. Chart.js renders charts such as stock by category, warehouse utilization and monthly stock movement. The barcode scanner API is used on the scan screen to read a product code from the device camera. Responsive styling allows the same interface to be used on computers and tablets.

3.2 Backend

The Node.js and Express.js backend exposes REST endpoints for authentication, products, suppliers, warehouses, locations, inventory, stock transactions, orders, barcode lookup, reports and slotting. Routes receive requests, middleware validates the token and role, and controller or service logic applies business rules. The mysql2/promise connection pool provides database access and supports multi-step transactions.

3.3 Database

The inventory_management MySQL database contains eleven related tables: users, staff, warehouses, locations, categories, suppliers, products, inventory, stock_transactions, orders and order_items. Primary keys identify records and foreign keys link related data. Unique constraints on email, SKU and barcode prevent duplicates, and CHECK constraints stop negative quantities and invalid capacities.

3.4 Version Control

Git and GitHub are used to track frontend, backend and database changes in a shared project repository. Environment secrets are kept outside source control.

3.5 Authentication and Security Libraries

JSON Web Tokens (jsonwebtoken) secure protected requests. Passwords are stored as salted hashes using bcryptjs rather than as plain text. Protected routes verify the token and the user's role before running. Only administrators can create manager and staff accounts, and all database queries use parameters to avoid SQL injection.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a standard three-tier architecture that separates the user interface, the business logic and the persisted data. Figure 4.1 shows the three layers and how a request flows between them.

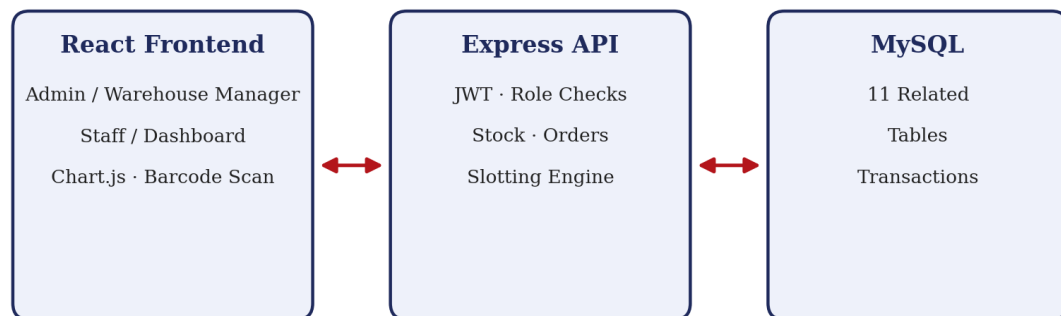


Figure 4.1: Inventory and Warehouse Management three-tier architecture

4.2 Description of Each Layer

The Presentation Layer is the React single-page application. It never talks to MySQL directly; every piece of data it shows comes from calling the REST API over HTTP.

The Application Layer is the Express REST API. It validates authentication and roles, checks product and stock availability, enforces location capacity, records stock transactions and runs the slotting algorithm. Database transactions connect related updates so that a failed operation does not leave partial data behind.

The Data Layer is the MySQL database, accessed through a connection pool rather than a single long-lived connection, so multiple requests can be served concurrently without waiting on each other unnecessarily.

4.3 Deployment Architecture

For local development, the Express backend is planned to listen on port 5000, the React frontend on port 5173 and MySQL on port 3306. The frontend sends API requests to the backend rather than connecting directly to the database. A hosted deployment would add HTTPS, managed secrets, a production database and monitoring.

CHAPTER 5: DESIGN

5.1 Database Design Principles

The schema is normalized to remove redundant data: each fact is stored in exactly one place, and every relationship between entities is expressed through a foreign key rather than by duplicating data across tables. Every table enforces its own invariants directly in the database, using PRIMARY KEY, FOREIGN KEY, UNIQUE and CHECK constraints, so that invalid data (a negative quantity, a duplicate SKU, a location with zero capacity) is rejected at the database level even if a bug elsewhere in the application tried to insert it.

5.2 ER Diagram / Database Schema

The database contains eleven related tables; Figure 5.1 shows the main relationships and Table 5.1 summarizes each table and its key fields.

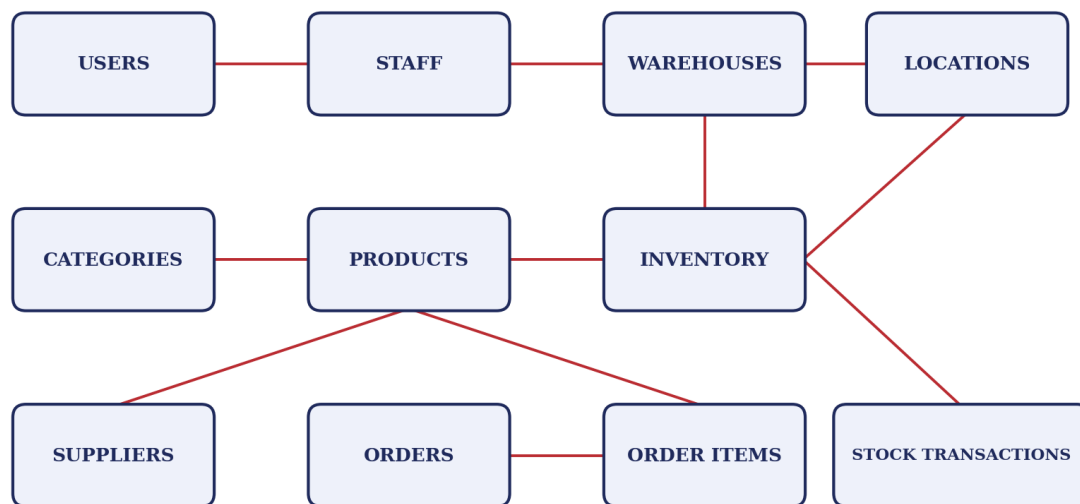


Figure 5.1: Main relationships of the 11-table database

Table	Purpose	Key fields
users	Login accounts and roles	user_id (PK), name, email (UNIQUE), password_hash, role
staff	Staff profile and assigned warehouse	staff_id (PK), user_id (FK), warehouse_id (FK), designation, phone
warehouses	Warehouse master data	warehouse_id (PK), name, address, total_capacity
locations	Storage slots inside a warehouse	location_id (PK), warehouse_id (FK), code, capacity, storage_type, distance_to_dock
categories	Product grouping	category_id (PK), name, description
suppliers	Supplier contact details	supplier_id (PK), name, email, phone, address
products	Product catalogue	product_id (PK), sku (UNIQUE), barcode (UNIQUE), name, category_id (FK), supplier_id (FK), unit_price, reorder_level, storage_type, demand_score

Table	Purpose	Key fields
inventory	Quantity of a product at a location	inventory_id (PK), product_id (FK), location_id (FK), quantity (CHECK >= 0)
stock_transactions	History of every stock movement	txn_id (PK), product_id (FK), location_id (FK), type, quantity, performed_by (FK), created_at
orders	Customer orders	order_id (PK), customer_name, status, created_by (FK), order_date
order_items	Products within an order	item_id (PK), order_id (FK), product_id (FK), quantity, unit_price

Table 5.1: Database tables and key fields

5.3 Design Decisions and Assumptions

- **Stock ledger:** quantities in the inventory table are changed only together with an insert into stock_transactions, inside one database transaction, so the current stock can always be explained by the history.
- **Capacity rules:** a location accepts stock only while its used quantity stays within its capacity and its storage_type matches the product's requirement.
- **Role-based access:** three roles are planned (Administrator, Warehouse Manager, Staff). Staff actions are limited to their assigned warehouse.
- **Slotting as a suggestion:** the optimization module proposes assignments; a manager reviews and applies them, and every move is recorded as a transaction.
- **Assumptions:** each location has a known distance from the dispatch dock, and demand is measured from recent order and stock-out history.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The application is divided into authentication, master data (products, categories, suppliers, warehouses, locations, staff), inventory and stock transactions, orders, barcode lookup, dashboard and reports, and warehouse slotting optimization.

Inventory and stock transactions. For stock-in, stock-out, transfer and adjustment, the backend starts a transaction, locks the affected inventory row with `SELECT ... FOR UPDATE`, checks that the quantity and location capacity allow the change, updates the inventory row and inserts a `stock_transactions` record, and then commits. If any step fails, the transaction rolls back. A transfer updates the source and destination rows in the same transaction.

Orders. When an order is created, each order item is checked against available stock and the quantities are deducted using the same transactional stock-out routine. Cancelling an order restores the stock and records the reversal.

Warehouse Slotting Optimization. The module decides where products should be stored so that the most requested products are the easiest to reach. Its steps are shown in Figure 6.1.



Figure 6.1: Warehouse Slotting Optimization workflow

- **Prioritize by demand:** each product receives a demand score from its recent units moved and order frequency. Products are placed in a priority queue (max-heap), so the highest-demand product is always taken first.
- **Order the locations:** free locations are sorted by distance to the dispatch dock, so the most accessible slots come first.
- **Greedy assignment:** for each product, taken in priority order, the algorithm picks the nearest location that has enough free capacity and the required storage type. It never exceeds capacity and records products that could not be placed.
- **Complexity:** sorting the products and locations costs $O(n \log n + m \log m)$ for n products and m locations. Matching costs up to $O(n \cdot m)$ in the worst case if locations are searched linearly, and less if locations are grouped by storage type.

Greedy assignment is fast and easy to explain, but it gives a good solution rather than a guaranteed global optimum; this is discussed in Chapter 10.

6.2 Front-end Logic

React state manages the logged-in user, selected warehouse, form inputs and loaded data. The dashboard shows summary cards (total products, low-stock items, warehouse utilization, pending orders) and Chart.js charts. Product and inventory tables support search, filtering and sorting. Stock-transaction forms validate quantities before submission, and loading, empty and error states help users understand pending requests. The scan screen reads a barcode through the camera and shows the matching product with its stock by location.

6.3 API Integration

The frontend sends JSON requests to the Express API under the /api prefix. Protected requests include the JWT in the Authorization header. The server returns product data, stock levels, transaction results and slotting suggestions, and the interface displays validation errors returned by the backend. A transaction is shown as successful only after the backend reports that the database commit succeeded.

6.4 Authentication and Authorization

Users log in with their email and password. The backend compares the password with the stored hash and returns a signed JWT that carries the user id and role. Middleware verifies the token on every protected route and checks the role: administrators manage users and master data, managers view reports and run slotting, and staff record stock movements and scan products. Staff accounts are linked to a warehouse, and the backend rejects requests for other warehouses. Accounts are created by an administrator; there is no public sign-up.

CHAPTER 7: FEATURES

7.1 List of Main Features

Administrator and manager features:

- Add and edit products, categories and suppliers, including barcode and SKU
- Create warehouses and storage locations with capacity and storage type
- Manage staff accounts and assign staff to warehouses (administrator)
- View products stored in each warehouse and the capacity used
- View dashboard charts and low-stock alerts
- Run Warehouse Slotting Optimization and review the suggested assignments

Warehouse staff features:

- Record stock-in, stock-out, transfer and adjustment transactions
- Scan a barcode to find a product and its stock by location
- Create orders and update order status
- View stock transaction history for the assigned warehouse

7.2 How Each Feature Works

Stock transactions: the user selects a product, location and quantity. The backend locks the inventory row, checks stock and capacity, updates the quantity and writes a transaction record. A rejected request returns a clear message and changes nothing.

Barcode lookup: the user scans or types a code, and the backend returns the product with its stock by warehouse and location, so staff can find where to pick or store it.

Dashboard: the backend aggregates stock, utilization and movement data, and Chart.js displays it as charts that managers can read at a glance.

Slotting optimization: the manager starts the optimization for a warehouse. The system scores products by demand, sorts the free locations by accessibility and proposes assignments using the greedy method. The manager reviews the proposal and applies it, and each move is stored as a transaction.

Screenshots of these screens will be added from the running application:

Screenshot to be added from the running application.

CHAPTER 8: TESTING

8.1 Postman API Testing

Postman can validate the authentication, product, inventory, transaction, order, scan and slotting endpoints. Login returns a JWT that is supplied to protected requests. A complete test flow logs in, creates a product and location, records stock-in, performs a stock-out and a transfer, creates an order and checks the resulting quantities. Invalid input and unauthorized requests should also be tested.

8.2 Integration Testing

Integration testing should verify the full workflow against MySQL: login, master-data setup, stock-in, transfer, order creation and cancellation, barcode lookup and slotting. Database records should be checked after each step. Concurrent stock-out requests for the last units of a product should also be tested. No execution log was available when this report was prepared.

8.3 Tools Used for Testing

- Postman: manual and scripted API testing
- curl (command line): scripted regression testing
- MySQL command-line client or Workbench: direct verification of table contents and constraint behaviour
- Manual browser testing: verifying the React frontend's behaviour and appearance

8.4 Test Cases and Results

The following cases describe the expected validation results. Actual results are marked Pending until recorded during execution of this project.

#	Test Case	Expected Result	Actual Result
1	Login with wrong password	401; login rejected	Pending
2	Request with expired or missing JWT	401; access denied	Pending
3	Staff calls an admin-only route	403 Forbidden	Pending
4	Add product with a duplicate SKU	409; duplicate rejected	Pending
5	Stock-out above available quantity	Rejected; stock never negative	Pending
6	Stock-in beyond location capacity	Rejected; capacity respected	Pending
7	Request a negative quantity	400; invalid input rejected	Pending
8	Transfer stock between locations	Both rows updated together	Pending
9	Transfer fails midway	Rolled back; no partial change	Pending
10	Two simultaneous stock-outs for the last units	Only one succeeds	Pending
11	Create an order with enough stock	Order saved; stock reduced	Pending
12	Cancel an order	Cancelled; stock restored	Pending
13	Scan a known and an unknown barcode	Product shown / not-found message	Pending
14	Run slotting optimization	High-demand products get nearest valid slots; no capacity exceeded	Pending

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

The application is planned to run locally for development and demonstration as follows. The folder names follow the planned repository layout and should be updated to match the final repository.

- Load the database: `mysql -u root -p < database/schema.sql`
- Start the backend: `cd backend, npm install, npm start (port 5000)`
- Start the frontend: `cd frontend, npm install, npm run dev (port 5173)`
- Open `http://localhost:5173` and log in with the administrator account

9.2 Environment Configuration

Configure `backend/.env` with `PORT`, the database host and port, `DB_USER`, `DB_PASSWORD` and `DB_NAME=inventory_management`, plus a private `JWT_SECRET`. The initial administrator account is created through a setup script using `ADMIN_NAME`, `ADMIN_EMAIL` and `ADMIN_PASSWORD`. Keep actual secrets outside source control.

9.3 Hosting of Frontend and Backend

The project is documented for local demonstration using a React frontend, an Express backend and a MySQL database. A production deployment requires a compiled frontend, an HTTPS API, protected environment configuration and regular database backups. The API base URL and CORS settings must match the hosted frontend.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- **Concurrent stock updates:** two users can try to issue the last units of a product at the same moment, which could make stock negative.
- **Capacity and storage rules:** stock must never exceed a location's capacity or be placed in an unsuitable storage type.
- **Balancing demand and accessibility:** the slotting module must trade off product demand, free capacity and distance, and the best slots are limited.
- **Barcode input:** camera-based scanning depends on lighting, device quality and browser permissions.
- **Access control:** staff must see and change only data for their own warehouse.

10.2 Solutions Applied

Row-level locking inside database transactions serializes competing stock updates, and CHECK constraints are a final safeguard against negative quantities. Capacity and storage-type checks run in the backend before any update. The slotting module uses a priority queue for demand and a sorted list of locations for accessibility, and skips locations that fail the capacity or type check. Manual barcode or SKU entry is kept as a fallback to scanning. Middleware checks the role and the assigned warehouse on every protected request.

10.3 Current Limitations or Known Bugs

- The greedy slotting method gives a good assignment quickly but does not guarantee the global optimum.
- Demand is estimated from past data and does not predict future demand.
- There is no integration with RFID readers, scanners with dedicated hardware drivers or automated storage equipment.
- Native Android or iOS distribution is outside the current React web implementation.
- Full execution evidence and cloud-deployment results were not available when this report was prepared and should be recorded before final evaluation.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- Automatic low-stock alerts and reorder suggestions by email or SMS.
- Demand forecasting from historical order data to improve slotting decisions.
- Password reset and account verification by email.
- A detailed audit log and downloadable PDF or Excel reports.

11.2 Possible Integrations or Optimizations

- Containerizing the application with Docker for a consistent, reproducible deployment.
- Stronger slotting algorithms that also consider picking routes and product affinity.
- RFID or handheld-scanner integration for faster stock counts.
- Caching frequently used reports and optimizing queries as data grows.
- Expanding automated API and concurrency tests, and adding monitoring for failed transactions.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

The Inventory and Warehouse Management System provides a database-driven approach to running a warehouse. The documented application combines a React frontend, an Express API and an eleven-table MySQL database to support product and supplier management, stock transactions, orders, barcode lookup, dashboard reporting and warehouse slotting optimization.

12.2 What Was Achieved

The project documentation covers secure role-based login, master-data management, transactional stock movement with capacity checks, order handling, barcode lookup, dashboard charts and a slotting module based on sorting, priority queues and a greedy algorithm. The validation plan records expected results; final execution evidence and screenshots should be added after testing the running application.

12.3 Skills Learned

The project develops practical skills in relational database design, REST API development, authentication and role checks, and React component development with Chart.js. It also shows why transactions and row locks matter when several users change stock at once, and how data-structure techniques such as sorting, heaps and greedy methods solve a real warehouse problem. Working as Team 17 helped the members coordinate implementation, documentation and demonstration tasks.

CHAPTER 13: REFERENCES

- React: Official Documentation, <https://react.dev>
- Chart.js: Official Documentation, <https://www.chartjs.org/docs/>
- Node.js: Official Documentation, <https://nodejs.org/en/docs>
- Express.js: Official Documentation, <https://expressjs.com>
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- mysql2 (Node.js MySQL driver): npm documentation, <https://www.npmjs.com/package/mysql2>
- jsonwebtoken: npm documentation, <https://www.npmjs.com/package/jsonwebtoken>
- bcryptjs: npm documentation, <https://www.npmjs.com/package/bcryptjs>
- Postman Learning Center, <https://learning.postman.com>
- MDN Web Docs: JavaScript and Web APIs reference, <https://developer.mozilla.org>
- T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms (sorting, heaps and greedy algorithms).

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippets

Illustrative greedy slotting routine (JavaScript):

```
// Highest-demand products get the most accessible valid slot
function assignSlots(products, slots) {
  const queue = [...products]
    .sort((a, b) => b.demandScore - a.demandScore);
  const free = [...slots]
    .sort((a, b) => a.distanceToDock - b.distanceToDock);
  const plan = [];
  for (const p of queue) {
    const i = free.findIndex(s =>
      s.freeCapacity >= p.unitsToStore &&
      s.storageType === p.storageType);
    if (i === -1) {
      plan.push({ productId: p.id, slotId: null });
      continue;
    }
    plan.push({ productId: p.id, slotId: free[i].id });
    free[i].freeCapacity -= p.unitsToStore;
    if (free[i].freeCapacity === 0) free.splice(i, 1);
  }
  return plan;
}
```

Illustrative transaction for issuing stock (SQL):

```
START TRANSACTION;
SELECT quantity FROM inventory
  WHERE product_id = ? AND location_id = ? FOR UPDATE;
-- Check that quantity >= requested before updating.
UPDATE inventory SET quantity = quantity - ?
  WHERE product_id = ? AND location_id = ? AND quantity >= ?;
INSERT INTO stock_transactions
  (product_id, location_id, type, quantity, performed_by)
  VALUES (?, ?, 'OUT', ?, ?);
COMMIT;
```

Appendix B: Installation and Setup Instructions

- `mysql -u root -p < database/schema.sql`
- `cd backend && npm run seed && npm run setup:admin`
- `cd backend && npm install && npm start (backend, port 5000)`
- `cd frontend && npm install && npm run dev (frontend, port 5173)`
- Open `http://localhost:5173` in a browser

Script names follow the planned project layout and should be adjusted to match the final repository.

Appendix C: Default Login Credentials

Role	How the account is created
Administrator	Configured through backend/.env and the admin setup script
Warehouse Manager	Created by the administrator from the Users screen
Staff	Created by the administrator and assigned to a warehouse
Password	Chosen at account creation and stored as a hash

No fixed shared login is required; credentials are never stored in the source code.

Appendix D: API Reference Summary

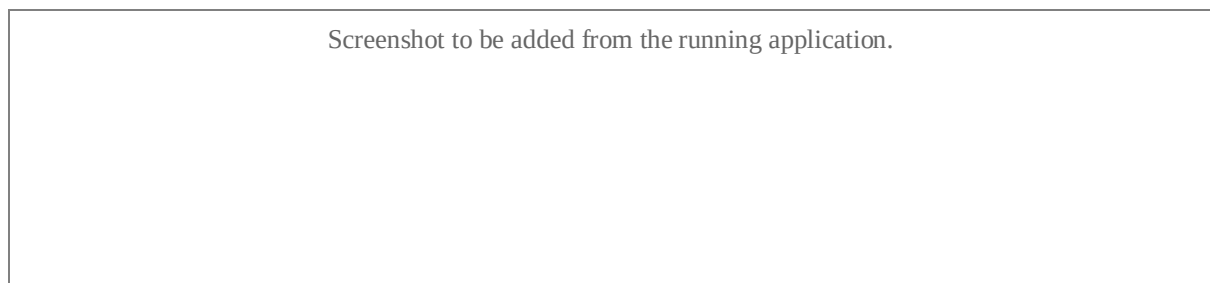
API resources cover authentication, master data, inventory, orders, scanning, reports and slotting. The backend uses the /api prefix.

Resource	Endpoints
Auth	Login and current-user access
Products	List, search, create, edit and delete products and categories
Suppliers	List, create and edit suppliers
Warehouses	Manage warehouses and storage locations; view stored products and capacity
Inventory	View stock by product, warehouse and location
Transactions	Record stock-in, stock-out, transfer and adjustment; view history
Orders	Create, list, update status and cancel orders
Scan	Look up a product by barcode or SKU
Reports	Dashboard summaries and chart data
Slotting	Generate and apply slotting suggestions

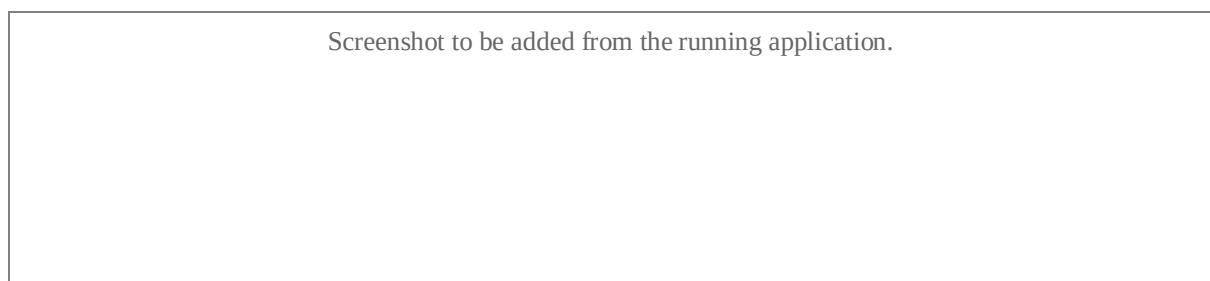
Appendix E: Screenshots

Screenshots are to be added from the running application.

1. Login Page



2. Dashboard



3. Product and Supplier Management

Screenshot to be added from the running application.

4. Warehouse and Location Capacity View

Screenshot to be added from the running application.

5. Stock Transactions

Screenshot to be added from the running application.

6. Order Management

Screenshot to be added from the running application.

7. Barcode Scan Page

Screenshot to be added from the running application.

8. Warehouse Slotting Optimization Result

Screenshot to be added from the running application.